Tutorial 5

Praktikum Pemrograman Berbasis Objek
Asisten IF2010 2025/2026

Daftar Isi

1. Interface

1. Generic

1. Wildcard

1. Collections

Interface

Why Interface?

Dalam OOP, inheritance membantu kita menjelaskan “object ini siapa”,

Contoh:

Mage is-a Character; Fighter is-a Character; Healer is-a Character

Tapi kadang kita juga peduli Object itu bisa melakukan apa

Contoh:

Mage → bisa menyerang	Fighter → bisa menyerang
Healer → bisa menyembuhkan	Paladin → bisa menyerang dan menyembuhkan

Kalau kemampuan tiap class berbeda-beda, apakah semua kemampuan harus dipaksa masuk ke parent class?

Why Interface?

Contoh kalau cuma pakai inheritance

```
class Character {
  private int hp;
  private int level;

  /* constructor, getter, setter */

  public void attack() { }
  public void heal() { }
}

class Mage extends Character { }
class Fighter extends Character { }
class Healer extends Character { }
class Paladin extends Character { }
```

Masalah di kode tersebut muncul, yaitu semua Character sekarang terlihat bisa **attack** dan **heal**, padahal Healer **gak bisa** attack, terus Mage **gak bisa** heal. Secara kode mungkin bisa jalan aja dengan dibikin kosong atau throw error, tapi model OOP-nya udah mulai ngaco alias cuma nutupin masalah desain.

Why Interface?

Jadi kita menginginkan bahwa:
Mage is-a Character, and can Attack.
Healer is-a Character, and can Heal.
Paladin is-a Character, and can Attack + Heal

Kita tetap butuh bahwa Mage, Healer, Paladin juga mewarisi atribut umum dari Character, seperti *hp* dan *level*. Tapi untuk kemampuan seperti *attack* dan *heal*, kita butuh mekanisme lain yang tidak memaksa semua subclass punya. Ingat juga bahwa di **Java**, kita **tidak bisa** melakukan **multiple inheritance**.

Di sinilah peran **interface** bisa membantu permasalahan seperti itu.

Interface

Attackable.java

```java
public interface Attackable {
  public void attack();
}
```

Healable.java

```java
public interface Healable {
  public void heal();
}
```

Mage.java

```java
class Mage extends Character implements Attackable {
  public void attack() {
    // implementasi method attack
  }
}
```

Paladin.java

```java
class Paladin extends Character implements Attackable, Healable {
  public void attack() {
    // implementasi method attack
  }
  public void heal() {
    // implementasi method heal
  }
}
```

Healer.java

```java
class Healer extends Character implements Healable {
  public void heal() {
    // implementasi method heal
  }
}
```

Dengan interface, kita gak perlu maksa semua ability masuk ke parent class

Interface

Terdapat suatu contoh permasalahan lain sebagai berikut

```
public static void criticalHit(??? unit) {
  unit.attack();
}
```

Pertanyaan: parameter unit di method tersebut harus bertipe apa?

Mage? Terlalu sempit. Masa Fighter gak bisa masuk juga
Character? Terlalu luas. Masa Healer juga ikut masuk
Object? Horee sekarang batu, kursi, nasi padang juga bisa masuk 😭

Kita butuh suatu tipe yang “apapun class-nya, **yang penting bisa attack**”

Interface

Jawabannya adalah kita gunakan tipe **Attackable**

```
public static void criticalHit(Attackable unit) {
  unit.attack();
}

criticalHit(new Mage());
criticalHit(new Fighter());
criticalHit(new Paladin());
```

Fungsi criticalHit tidak peduli object aslinya apa, yang penting object itu punya kemampuan attack().

Jadi interface itu → **Fleksibel** dan **Aman**

Fleksibel: Banyak class berbeda bisa diproses dengan cara yang sama.

Aman: Java memastikan semua class yang masuk benar-benar punya method yang dibutuhkan.

Perbedaan dengan C++

Di **C++**, kalian mungkin memakai **abstract class/pure virtual function** untuk membuat kontrak perilaku.

Di **Java**, **interface** adalah cara yang lebih eksplisit dan umum untuk mendefinisikan kontrak perilaku. Yang namanya kontrak, berarti ketika suatu class implements interface tersebut, harus menerapkan semua hal yang tertulis di interface tersebut.

Generics

Why Generics?

Kadang kita ingin membuat class atau method yang logic-nya sama, tapi tipe datanya bisa berbeda-beda.

Contoh:

Box untuk String	Stack untuk Integer
Box untuk Integer	Stack untuk String
Box untuk Character	Stack untuk Item

Masalahnya, apakah kita harus buat class baru untuk setiap tipe data?

Why Generics?

```java
// contoh buruk
class StringBox {
    private String value;

    public void setValue(String newVal){
        value = newVal;
    }

    public String getValue() {
        return value;
    }
}

class IntegerBox {
    private Integer value;

    public void setValue(Integer newVal){
        value = newVal;
    }

    public Integer getValue() {
        return value;
    }
}
```

Isi classnya hampir sama, yang beda cuma tipe datanya. Kalau tiap tipe butuh class sendiri, ini udah gak OOP lagi, bakal bikin capek copas

Why Generics?

Kalau pake Object?

```
// contoh setengah benar
class Box {
    private Object value;

    public void setValue(Object newVal){
        value = newVal;
    }

    public Object getValue() {
        return value;
    }
}

Box box = new Box();
box.setValue("Abc123");
String kata = (String) box.getValue();

box.setValue(123);
String kata2 = (String) box.getValue(); //runtime✕
```

Object emang fleksibel, tapi terlalu fleksibel sampai type safety hilang.

Generics

Generic solusinya

```
// contoh benar
class Box<T> {
    private T value;

    public void setValue(T newVal){
        value = newVal;
    }

    public T getValue() {
        return value;
    }
}

Box<String> stringBox = new Box<>();
stringBox.setValue("Abc123");
String kata = stringBox.getValue();

Box<Integer> integerBox = new Box<>();
integerBox.setValue(100);
Integer score = integerBox.getValue();
```

Generic bikin kode reusable tanpa mengorbankan type safety.

Generics

Analogi Blender Generic

```
class Jus<T> {
    private T bahan;

    public Jus(T bahan){
        this.bahan = bahan;
    }

    public T getBahan() {
        return bahan;
    }
}

Jus<Apel> jusApel = new Jus<>(new Apel());
Jus<Sayur> jusSayur = new Jus<>(new Sayur());
Jus<Bumbu> jusBumbu = new Jus<>(new Bumbu());
```

Blendernya satu, tipe bahannya bisa beda-beda. Tapi **sekali dipilih**, **tipenya** tetap **dijaga**

Generics

Analogi Blender Generic

```
class Jus<T> {
    private T bahan;
    ...
    public void setBahan(T bahan) {
        this.bahan = bahan;
    }
}

Jus<Apel> jusApel = new Jus<>(new Apel());

jusApel.setBahan(new Apel());  // OK ✅
jusApel.setBahan(new Sayur()); // ERROR ❌
```

Generic itu fleksibel saat tipenya dipilih, tapi ketat setelah tipe ditentukan.

Perbedaan dengan C++

Mirip Template C++, tapi tidak sama

```
// C++
template<class T>
class Box {
private:
    T value;

};

// Java
class Box<T> {
    private T value;
};
```

Java Generic lebih fokus pada type safety di compile-time

Generic Method

Kadang yang perlu generic bukan satu class full, tapi hanya satu method.

```
public class Element {
    public static <T> void printArray(T[] array) {
        for (T item : array) {
            System.out.println(item);
        }
    }
}

String[] names = {“Ayu”, “Budi”};
Integer[] scores = {90, 85, 100};

Element.printArray(names);   // pemanggilan type inference
Element.printArray(scores);  // Java menebak tipe otomatis
Element.<Integer>printArray(scores); // pemanggilan explicit
```

<T> sebelum return type menandakan bahwa method ini punya type parameter sendiri

Generic Method: Max Element of Array

```cpp
// element.cpp
template<class T>
T max_elmt(T* arr, int N)
{
    T max_result = arr[0];
    for (int i = 0; i < N; i++) {
        if (max_result < arr[i]) {
            max_result = arr[i];
        }
    }
    return max_result;
}
```

```java
// Element.java
public class Element {
    public static <T> T max_elmt(T[] arr, int N)
    {
        T max_result = arr[0];
        for (int i = 0; i < N; i++) {
            if (max_result < arr[i]) { // ERROR
                max_result = arr[i];
            }
        }
        return max_result;
    }
}
```

Di Java, operator binary hanya dapat digunakan pada tipe primitif saja (`int`, `float`, dll.)
Untuk Object bisa pakai compareTo() atau membatasi T ke tipe tertentu.

Java tahu bahwa T adalah suatu tipe, tapi belum tahu apakah T bisa dibandingkan.

Bounded Generic

Bagaimana kalau T kita coba batasi tidak boleh sembarang tipe?

Contoh:

Jus<T> → boleh jadi Jus<Apel>, Jus<Sawi>, Jus<Bumbu>

Tapi kalau kita ingin khusus jus buah?

```
class JusBuah<T extends Buah> {
    private T buah;

    public JusBuah(T buah){
        this.buah = buah;
    }

    public T getBuah() {
        return buah;
    }
}
```

```
JusBuah<Apel> jusApel = new JusBuah<>(new Apel());       // OK ✅
JusBuah<Mangga> jusMangga = new JusBuah<>(new Mangga()); // OK ✅
JusBuah<Sawi> jusSawi = new JusBuah<>(new Sawi());       // ERROR ❌
```

T extends Buah artinya:

T boleh tipe apapun, asalkan Buah atau turunannya.

“extends” bisa dipakai untuk class dan interface

Multiple Bound

Kadang kita gak cukup hanya bilang: “T harus turunan dari Buah”

Kita juga ingin bilang: “T harus Buah, bisa dicuci, dan bisa disimpan”

```
class JusMaker<T extends Buah & Washable & Storable> {
    public void makeJuice(T buah) {
        buah.wash();
        buah.store();
        System.out.println(“Membuat jus dari “ + buah.getName());
    }
}
```

Di Java, kalau multiple bound berisi class dan interface, maka **class harus ditulis dulu**.

Common Mistakes

1. Pakai primitive type

```
ArrayList<Integer> numbers;  // OK ✓
ArrayList<int> numbers;      // ERROR ✗
```

1. Raw type

```
ArrayList list = new ArrayList(); // Avoid
list.add("Hello");
list.add(123);
```

→

```
ArrayList<String> list = new ArrayList<>();
list.add("Hello"); // OK ✓
list.add(123);     // ERROR ✗
```

1. Mengira generic itu sama dengan Object

```
Box<Object> box = new Box<String>();  // ERROR ✗
```

Box<String> bukan subtype dari Box<Object>, walaupun String adalah subtype dari Object

Masalah Baru

Kadang generic masih terlalu ketat

```
List<Integer> integers = new ArrayList<>();
List<Number> numbers = integers; // ERROR ✕
numbers.add(3.14); // bahaya
```

Padahal kan Integer itu Number, kenapa List<Integer> bukan List<Number>?

Karena kalau diizinkan, kita bisa memasukkan Double ke List<Integer>

Untuk membuat generic lebih fleksibel tanpa merusak type safety,
Java menyediakan **WILDCARD**.

Wildcard

Why Wildcard?

Mengingat problem List<Integer> dengan List<Number>, kita butuh sesuatu yang fleksibel lagi tapi tetap aman. Misal kita ingin method yang bisa menerima:
List<Integer>, List<Double>, List<Float>

Selama semua elemennya masih keluarga Number

```
public static void printNumbers(??? numbers) {
    for (Number number : numbers) {
        System.out.println(number);
    }
}
```

Parameter numbers harus bertipe apa?

Jawabannya adalah **Wildcard** →

```
public static void printNumbers(List<? extends Number> numbers) {
    for (Number number : numbers) {
        System.out.println(number);
    }
}
```

Wildcard

Wildcard adalah tipe yang belum diketahui.
Contoh: List<?> items
Artinya: List berisi suatu tipe tertentu, tapi kita tidak tahu tipe spesifiknya apa.
Bisa menunjuk ke List<String>, List<Integer>, List<Item>.

Tapi karena tipe pastinya tidak diketahui, kita tidak boleh sembarang memasukkan elemen baru

Unbounded Wildcard

Kita kembali ke kasus jus, tapi kali ini adalah
Jus<?> jusMisterius;
Artinya: Jus ini dari suatu bahan tertentu, tapi kita tidak tahu bahan spesifiknya apa.

Intinya Wildcard bukan berarti campur bebas, hanya saja tipe spesifiknya tidak diketahui.

Upper-Bounded Wildcard

Upper-Bounded simbolnya: <? extends T>

Contoh:

```
List<? extends Number> numbers;
// List tersebut berisi suatu tipe yang
merupakan Number atau turunannya
List<Integer>
List<Double>
List<Float>
List<Number>
// Tapi tidak bisa menerima
List<String>
List<Object>
```

```
public static double sum(List<? extends Number> numbers) {
    double total = 0;
    for (Number number : numbers) {
        total += number.doubleValue();
    }
    return total;
}
```

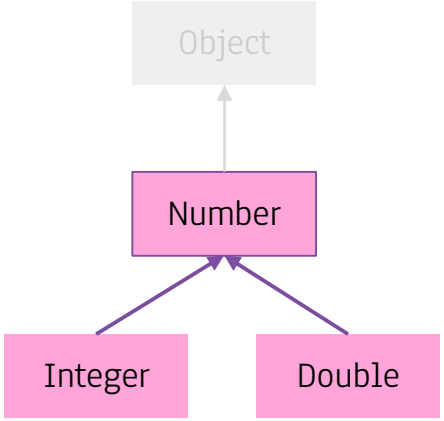
Sekarang kalau pemanggilan:

```
List<Integer> ints = Arrays.asList(1, 2, 3);
List<Double> doubles = Arrays.asList(1.5, 2.5, 3.5);

sum(ints);    // OK ✅
sum(doubles); // OK ✅
```

Upper-Bounded Wildcard

Akan tetapi

```
public static void addNumber(List<? extends Number> numbers) {
    numbers.add(10);    // ERROR ✕
    numbers.add(3.14);  // ERROR ✕
}
```



Loh, katanya extends Number?

Integer dan Double kan Number?

Kenapa gak boleh add?

Karena Java tidak tahu tipe asli list tersebut.

```
List<? extends Number> numbers = new ArrayList<Integer>();
numbers.add(3.14) // bahaya kalau aslinya List<Integer>
numbers.add(10) // bahaya juga kalau misal aslinya List<Double>
```

? extends T aman untuk membaca, tidak aman kalau nambah elemen baru

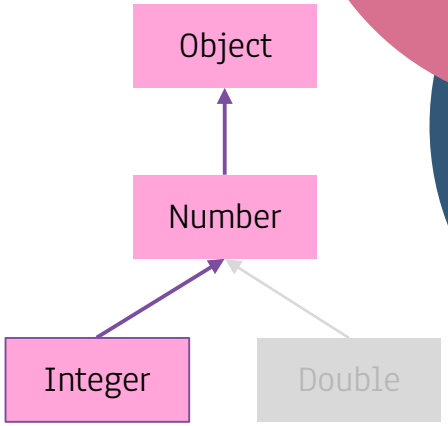
Lower-Bounded Wildcard

Lower-Bounded simbolnya: <? super T>

Contoh:

```
List<? super Integer> numbers
// List tersebut berisi suatu tipe yang merupakan Integer atau
parent dari Integer
List<Integer>
List<Number>
List<Object>
// Tapi tidak bisa menerima
List<Double>
List<String>
```

```
public static double addIntegers(List<? super Integer> numbers) {
    numbers.add(1);
    numbers.add(2);
    numbers.add(3);
}
```



Pemanggilan

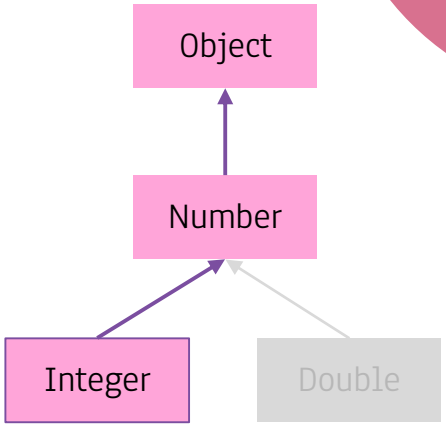
```
List<Integer> integers = new ArrayList<>();
List<Number> numbers = new ArrayList<>();
List<Object> objects = new ArrayList<>();

addIntegers(integers); // OK ✓
addIntegers(numbers); // OK ✓
addIntegers(objects); // OK ✓
```

Lower-Bounded Wildcard

Akan tetapi

```
Integer x = list.get(0); // ERROR ✗
Objects o = list.get(0); // OK ✓
```

Karena list tersebut mungkin sebenarnya List<Object>, yang isinya bisa string, Double, atau object lain.



? super T aman untuk menulis T, tapi saat membaca, yang aman hanya Object

PECS: Producer Extends, Consumer Super

Producer Extends: Kalau collection menghasilkan data untuk kita baca, pakai ? extends T.

Consumer Super: Kalau collection menerima data yang kita masukkan, pakai ? super T.

Generic vs Wildcard

GENERIC TYPE	WILDCARD
Dapat memiliki >1 bound	Hanya dapat memiliki satu bound
Tidak ada lower bound, hanya upper bound (extends)	Ada lower bound (super) dan upper bound (extends)
Dapat digunakan ketika meng-enforce tipe yang berhubungan `public static <T extends Number> void copy(List<T> dest, List<T> src)`	Tidak dapat digunakan untuk meng-enforce tipe yang berhubungan `public static void copy(List<?> dest, List<?> src) // tipe isi List dest dan src dapat berbeda`

Tipe dapat digunakan di-refer kembali pada method body
`public <T extends Number> Map<T, String> convertToMap(ArrayList<T> list) {`
`  Map<T, String> names = new HashMap<T, String>();`
`  ...`
`}`
	Tipe tidak dapat digunakan di-refer kembali pada method body

Rangkuman Analogi Jus

Jus<T> → blender fleksibel, tipe bahan ditentukan saat dipakai

Jus<T extends Buah> → blender khusus buah, boleh Apel, Mangga, Kiwi

Jus<?> → tipe bahan tidak diketahui, jadi tidak bisa dipakai ulang secara spesifik

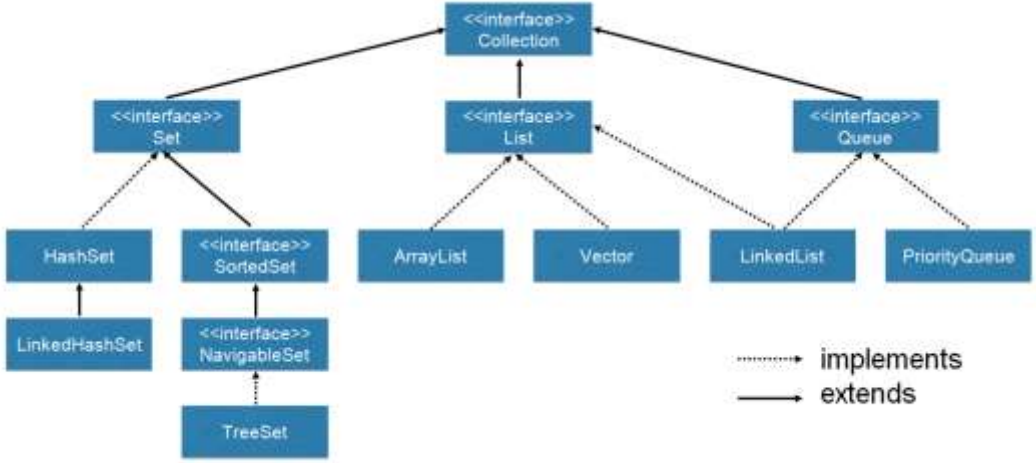
Jus<? extends Buah> → jus buah misterius, tapi pasti dari Buah tapi gak tahu spesifiknya Buah apa.

Jus<? super Apel> →wadah yang pasti bisa menerima Apel, bisa wadah Apel, wadah Buah, atau wadah Object

Java API: Collections

Collection

- Terdapat Collection interface dalam Java API yang dapat digunakan untuk mengimplementasikan sebuah kumpulan data



List

```
import java.util.*;

public class Shuffle {
   public static void main(String[] args) {
            List<String> list = new ArrayList<String>();
            for (String a : args)
         list.add(a);
            Collections.shuffle(list, new Random());
            System.out.println(list);
   }
}
```

List adalah Collection yang urutan elemennya disesuaikan dengan urutan pemasukannya.

Implementasi List:

- ArrayList
- LinkedList

Queue

```java
import java.util.*;

public class Countdown {
    public static void main(String[] args) throws InterruptedException {
        int time = Integer.parseInt(args[0]);
        Queue<Integer> queue = new LinkedList<Integer>();
        for (int i = time; i >= 0; i--)
            queue.add(i);
        while (!queue.isEmpty()) {
            System.out.println(queue.remove());
            Thread.sleep(1000);
        }
    }
}
```

Queue adalah Collection yang menyimpan elemen sebelum preprocessing, umumnya bersifat FIFO.

Deque

```java
import java.util.*;

public class DequeExample {
  public static void main(String[] args){
    Deque<String> deque = new
LinkedList<String>();

    deque.add("Element 1");         // Tail
    deque.addFirst("Element 2");    // Head
    deque.addLast("Element 3");     // Tail
    deque.push("Element 4");        // Head
    deque.offerFirst("Element 5"); // Head
    deque.offerLast("Element 6");  // Tail
```

```java
    // Pop returns the head, and removes it
from the deque
    System.out.println("Pop " + deque.pop());
    System.out.println("After pop: " + deque);

    // We can remove the first / last element.
    deque.removeFirst();
    deque.removeLast();
    System.out.println("Deque after removing "
+ "first and last: " + deque);
    }
}
```

Deque (*double-ended queue*) adalah Collection yang elemennya dapat diakses dari HEAD dan TAIL.

Set

```java
import java.util.*;

public class FindDups {
  public static void main(String[] args) {
    Set<String> s = new HashSet<String>();
    for (String a : args)
      s.add(a);
    System.out.println(s.size() + " distinct words: " + s);
   }
}
```

Set adalah Collection yang tidak bisa memiliki elemen duplikat.

Implementasi Set:

- HashSet
- TreeSet
- LinkedList

Map

```java
import java.util.*;

public class Freq {
    public static void main(String[] args) {
        Map<String, Integer> m = new HashMap<String, Integer>();
        // Initialize frequency table from command line
        for (String a : args) {
            Integer freq = m.get(a);
            m.put(a, (freq == null) ? 1 : freq + 1);
        }
        System.out.println(m.size() + " distinct words:");
        System.out.println(m);
    }
}
```

Map adalah Collection yang memetakan suatu *key* ke *value*.

Implementasi Map:

- HashMap
- TreeMap
- LinkedHashMap

General Purpose

General-purpose Implementations

Interfaces	Implementations				
	Hash table	Resizable array	Tree	Linked list	Hash table + Linked list
Set	HashSet		TreeSet		LinkedHashSet
List		ArrayList		LinkedList	
Queue					
Map	HashMap		TreeMap		LinkedHashMap

Java Stream API

Apa itu Java Stream API

- Java Stream API memungkinkan pengguna untuk melakukan operasi secara functional-style.
- Pandanglah setiap item dalam collection sebagai sebagai aliran/stream data yang ingin diproses!
- Secara umum terdapat empat proses:
  - filter()
  - map()
  - reduce()
  - forEach()
- Dapat memanfaatkan Java Lambda Expression. `(x, y) -> x + y`

Reduce

```java
// Contoh penggunaan stream API reduce

import java.util.*;

class ReduceMain {
  public static void main(String[] args) {
            List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);

            // Total semua elemen dalam list
            int result = numbers
            .stream()
            .reduce(0, (subtotal, element) -> subtotal + element);

            // Hasilnya 21
            System.out.println(result);
  }
}
```

For Each

```java
// Contoh penggunaan stream API forEach

import java.util.*;

class ForEachMain {
  public static void main(String[] args) {
            List<String> names = Arrays.asList("Larry", "Steve", "James");

            // Mencetak setiap nama dalam names
            names.forEach(System.out::println);

            // Keluarannya:
            // Larry
            // Steve
            // James
  }
}
```

Dokumentasi Java

- https://docs.oracle.com/javase/8/docs/api/

Sekian.

Ditunggu praktikum dan tutorial selanjutnya.